A MINIMAL FIELD HANDBOOK

The Minimal Inference Optimization Handbook

KV cache, batching, quantization and speculative decoding - serving LLMs at the memory wall.

Mosafer

vLLM · FlashAttention · GQA · speculative decoding

V1.0 - COMPACT EDITION

NOVEMBER 2026

Contents

1 Orientation **2**
- 1.1 The cost model . . . 2
- 1.2 The metric ladder . . . 2

2 The KV Cache **3**
- 2.1 The algebra . . . 3
- 2.2 Paged attention . . . 3
- 2.3 The eviction and sharing frontier . . . 3

3 Batching **5**
- 3.1 Static vs. continuous . . . 5
- 3.2 The arithmetic of batching . . . 5
- 3.3 Chunked prefill . . . 5

4 Speculative Decoding **7**
- 4.1 The algorithm . . . 7
- 4.2 Choosing the draft . . . 7

5 Attention Optimizations **9**
- 5.1 FlashAttention: exact, IO-aware . . . 9
- 5.2 GQA and MQA: fewer KV heads . . . 9
- 5.3 The length wall: windows and more . . . 9

6 Serving Stacks **11**
- 6.1 The engine menu . . . 11
- 6.2 Multi-GPU: sharding and replication . . . 11
- 6.3 The deployment patterns . . . 11

A Cheat Sheet **13**
- A.1 The cost model . . . 13
- A.2 KV cache arithmetic . . . 13
- A.3 Engine flags . . . 13
- A.4 Speculative decoding . . . 13
- A.5 Decision table . . . 14

CHAPTER 1

Orientation

LLM inference is two different problems wearing one API. The *prefill* (processing the prompt) is compute-bound — big matrix multiplies, GPU-friendly. The *decode* (generating tokens) is memory-bandwidth-bound — every token re-reads every weight to produce one number. Every optimization in this handbook is a move against one of those two walls.

1.1 The cost model

$$\text{prefill FLOPs} \approx 2ND_{\text{prompt}}, \qquad \text{decode per token} \approx 2N \text{ FLOPs}$$

with N parameters and D prompt tokens. Decode reads all N parameters per token: at bf16, a 7B model moves 14 GB per token — a GPU with 1 TB/s bandwidth caps at ~70 tok/s *no matter how fast the compute is*. Memory bandwidth is the decode wall; compute is the prefill wall.

Lever	Which wall it attacks
quantization (4-bit weights)	decode bandwidth: 4x fewer bytes moved (Open-Weights handbook, Ch. 4)
batching	decode bandwidth: one weight-read serves many sequences
speculative decoding	decode wall: several tokens per weight-read
KV-cache management	memory capacity: longer contexts, more concurrent requests
FlashAttention	prefill compute + attention memory

1.2 The metric ladder

Metric	Definition
TTFT	time to first token — prefill latency, felt as responsiveness
TPOT / tok/s	time per output token — the decode wall's clock
throughput	total tokens/sec across all users — the batching prize
goodput	throughput at a latency SLO — what production optimizes

```
import time, torch   # measure your own wall before optimizing
t0 = time.perf_counter()
out = model.generate(**inputs, max_new_tokens=256, do_sample=False)
dt = time.perf_counter() - t0
print(f"tok/s: {256 / dt:.1f}")        # single-stream decode speed
```

PITFALL

The two most common misdiagnoses: (1) "the GPU is idle during decode, so it must be compute-limited" — it is idle because it is *bandwidth*-limited; more FLOPs will not help, fewer bytes will; (2) comparing single-stream tok/s between engines and concluding one is "faster" — batching changes everything (Ch. 3); benchmark the workload you actually serve.

CHAPTER 2

The KV Cache

The KV cache is the state of generation: keys and values of every previous token, kept so each new token attends to history without recomputing it. It is also the memory bill that decides how long and how wide your serving can be.

2.1 The algebra

For each layer, each head, token t stores a key and a value vector of size d_{head}:

$$\text{KV bytes} = 2\ (\text{K+V}) \times L_{\text{ctx}} \times n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times \text{batch} \times \text{bytes per elem}$$

```python
def kv_gb(layers, kv_heads, d_head, ctx, batch=1, dtype=2):
    return (2 * layers * kv_heads * d_head * ctx * batch
            * dtype) / 1e9

# Llama-3-8B: 32 layers, 8 KV heads (GQA), 128 d_head
print(f"{kv_gb(32, 8, 128, 8192, batch=1):.1f} GB")   # ~1.1 GB @ bf16
print(f"{kv_gb(32, 8, 128, 8192, batch=32):.1f} GB")  # ~34 GB @ batch 32
```

GQA (fewer KV heads than query heads) is the architectural fix that made long context affordable — 8 KV heads instead of 32 is a 4x cut, with negligible quality loss.

2.2 Paged attention

Naive caching reserves max-context per sequence — contiguous, mostly-empty memory. vLLM's PagedAttention stores the cache in fixed blocks with an indirection table (virtual memory for the KV cache):

- memory waste drops from 60–80% (fragmentation + reservation) to under 4%;
- sequences share prefix blocks — *prefix caching* makes shared system prompts free after the first request;
- copy-on-write semantics for beam search and branching.

```
vllm serve Qwen/Qwen2.5-1.5B-Instruct \
    --enable-prefix-caching \
    --block-size 16 \
    --gpu-memory-utilization 0.9    # KV pool share
```

2.3 The eviction and sharing frontier

- **sliding-window / streaming attention**: cap the cache at W tokens; quality holds for tasks that do not need deep history.
- **cache eviction policies**: H2O-style attention-score eviction keeps the tokens that matter.
- **cross-request sharing**: same system prompt across users — prefix caching turns repeated prefills into one.

> **PITFALL**

Three KV-cache traps: (1) forgetting that decode batch memory is dominated by KV, not weights — OOM at batch 32 when weights are 14 GB and KV is 34 GB is the arithmetic, not a bug; (2) left-padding for batched generation (HF handbook, Ch. 6's pitfall) misaligns every position and silently corrupts attention; (3) multi-turn chats that re-send the full history per turn without prefix caching pay the same prefill over and over — enable prefix caching or manage sessions server-side.

THE ONE-SENTENCE SUMMARY

The KV cache is why context is the memory currency of serving: GQA and multi-bit quantization shrink it per token, paging removes the waste, prefix caching removes the repeats, and `-max-model-len` is the dial that decides how much of the GPU is a cache.

CHAPTER 3

Batching

Decode is bandwidth-bound, and a batch of B sequences moves the same weights as one — throughput scales with B almost for free. Batching is the single largest serving optimization, and its modern form (continuous batching) is what production engines sell.

3.1 Static vs. continuous

- **static batching**: collect B requests, run them all to their longest output — short sequences wait for the long one (padding waste per batch, often 2x).
- **continuous batching** (Orca): at every decode step, finished sequences leave and queued requests join — the GPU stays saturated without batch-boundary stalls. This is vLLM/TGI's core and the reason their throughput numbers look absurd next to `transformers`.

```
import concurrent.futures as cf
# vLLM: continuous batching is the engine default; just send concurrent
# requests (vllm serve Qwen/Qwen2.5-1.5B-Instruct --max-num-seqs 64):

with cf.ThreadPoolExecutor(32) as pool:
    outs = list(pool.map(lambda q: query(q), questions))
```

3.2 The arithmetic of batching

Decode per step: $\approx 2N$ FLOPs + one weight read of $N \cdot b_{\text{pw}}$ bytes, amortized over B sequences — tok/s per request barely drops until KV memory saturates:

Constraint	Binds when
weights read	always (the free lunch region: small B)
KV cache capacity	$B \times L_{\text{ctx}} \times$ kv_bytes vs. the KV pool (Ch. 2)
compute	large B: per-step FLOPs $= B \times 2N \times d_{\text{per-token}}$ catch up

Find your knee empirically: sweep `-max-num-seqs` and watch goodput, not throughput — throughput can rise while per-request latency violates the SLO.

3.3 Chunked prefill

Long prompts stall decode: prefilling a 30k-token prompt in one go blocks every other sequence for the whole pass. Chunked prefill splits prefill into compute chunks interleaved with decode steps — TTFT for the big request, TPOT stability for everyone else. Both vLLM and TGI expose it; enable it when workloads mix long prompts with chat.

> **PITFALL**
>
> Batching pitfalls: (1) padding waste — requests of wildly different lengths in static batches pad to the max; continuous batching solves it; (2) KV memory is the batch ceiling — `max_num_seqs` is bounded by Ch. 2's arithmetic, not by ambition; (3) fair-queueing — long prompts jumping the queue inflate everyone's TPOT; SLO-aware schedulers exist because this is real.

THE ONE-SENTENCE SUMMARY

Decode reads weights once per step no matter how many sequences share the step — so batch until memory or latency stops you, use continuous batching to never stall at batch boundaries, and chunk prefills so one long prompt does not tax the fleet.

CHAPTER 4

Speculative Decoding

Speculative decoding breaks the one-token-per-weight-read law: a cheap draft model guesses several tokens, the big model verifies them in *one* pass, and accepted tokens cost nothing extra. Same output distribution, 2–3x the speed.

4.1 The algorithm

1. draft model q (1–3B) generates k tokens greedily or sampled.
2. target model p scores *all* k draft tokens in one forward pass (they are just a longer prompt).
3. accept/reject each draft token by the test: accept with probability $\min(1, p(x)/q(x))$ — and if rejected, resample from the normalized residual $(\max(p - q, 0))$.
4. the modified-rejection construction guarantees the output is *exactly* distributed as p — lossless, no quality knob.

Expected tokens per target pass: $\frac{1-\alpha^{k+1}}{1-\alpha}$ where α is the average acceptance rate — with $\alpha = 0.8$ and $k = 4$: ≈ 3 tokens per weight-read.

```
from transformers import AutoModelForCausalLM, AutoTokenizer

target = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-7B-Instruct", device_map="auto")
draft = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-1.5B-Instruct", device_map="auto")
tok = AutoTokenizer.from_pretrained(name)

out = model.generate(**inputs, max_new_tokens=256,
                     assistant_model=draft,      # the whole feature
                     do_sample=True)
```

In transformers, pass `assistant_model=draft` to `generate()`; in vLLM, `-speculative-model` plus `-num-speculative-tokens`; in llama.cpp, `-draft-model`.

4.2 Choosing the draft

Draft	Trade-off
same family, smaller (1.5B for 7B)	shares the tokenizer; α is naturally high (0.7–0.9)
self-speculation (Eagle/Medusa heads)	extra heads on the target itself; tuned per model, best α
n-gram / prompt-lookup	free for copy-heavy tasks (summarization, editing): draft from the prompt itself

PITFALL

Speculative decoding's economics: the draft must be $\geq$ 5–10x cheaper per token than the target, or the verification overhead eats the win — a 1B draft for a 7B target is the sweet spot; two similar-size models is a slowdown. Acceptance rate is task-dependent: code and structured text accept well, high-temperature creative sampling less so — benchmark with your real temperature.

THE ONE-SENTENCE SUMMARY

The decode wall is bytes-per-token; speculative decoding amortizes the read across several tokens whose correctness is verified exactly — the output distribution is the target model's, the speed is closer to the draft's. Lossless acceleration exists; it just needs a small friend.

CHAPTER 5

Attention Optimizations

Attention's $O(L^2)$ score matrix and its memory traffic are the prefill wall and the long-context wall. FlashAttention removed the memory wall; GQA/MQA removed the KV wall; sliding windows removed the length wall. This chapter is the map.

5.1 FlashAttention: exact, IO-aware

The insight is not an approximation — it is *memory layout*. Naive attention materializes the $L \times L$ score matrix in HBM (slow GPU memory); FlashAttention computes attention in SRAM tiles with an online softmax (running max and sum), never writing the full matrix:

- same exact output, $O(L)$ memory instead of $O(L^2)$;
- wall-clock 2–4x faster in practice (IO-bound operation made compute-bound);
- enables long context — 128k contexts exist because of this line of work.

Use it by flag, not by code: `attn_implementation="flash_attention_2"` in `from_pretrained`, or the v3 kernels via the engine flags. Requirement: Ampere-or-newer, fp16/bf16.

5.2 GQA and MQA: fewer KV heads

Multi-Query Attention (MQA): all query heads share *one* KV head — KV cache and bandwidth shrink by hx (32x for 32 heads), with a small quality cost. Grouped-Query Attention (GQA): g KV heads for h query heads — the middle point that won (Llama 3, Qwen, Mistral: $g = 8$). The math is Ch. 2's KV formula; GQA is the n_{kv} dial.

Variant	KV heads	KV memory vs. MHA
MHA	h (e.g. 32)	1x (baseline)
GQA-8	8	4x smaller
MQA	1	32x smaller

5.3 The length wall: windows and more

- **sliding-window attention**: each token attends to the last W only — cache and compute become $O(L \cdot W)$; Mistral's 4096-window with stacked layers gives effective receptive fields beyond W (information passes layer to layer).
- **long-context tricks**: RoPE scaling (`rope_theta`, YaRN), positional interpolation — the position embedding is stretched so pre-trained models extend to longer contexts; quality degrades gracefully, verify on needle-in-haystack tests.
- **sparse/linear attention**: research-active; production still runs dense-plus-window.

> **PITFALL**
>
> Three attention-optimization traps: (1) FlashAttention requires contiguous heads and matched dtypes — falling back silently to the naive kernel halves your speedup; check the warning log; (2) GQA conversion of a fine-tune must map query heads to their group's KV head consistently — wrong grouping breaks the model subtly (the converters handle it; hand-rolled state-dict surgery often does not); (3) sliding-window

models mis-answer questions referencing text beyond the window — needle tests must place needles inside W, or use the stacked-layer receptive-field argument with evidence.

THE ONE-SENTENCE SUMMARY

FlashAttention says the quadratic score matrix never needs to touch slow memory; GQA says the cache does not need one KV per query head; windows say the cache does not need all history. Together they turned long-context serving from a research topic into a pricing page.

CHAPTER 6

Serving Stacks

The engines bundle every previous chapter into one flag. This chapter compares them and shows the deployment patterns: single GPU, multi-GPU sharding, and scale-out.

6.1 The engine menu

Engine	Strengths
vLLM	PagedAttention, continuous batching, speculative decoding, guided JSON — the GPU default
SGLang	RadixAttention (prefix-tree cache) + fast constrained decoding; strong for agent workloads
TensorRT-LLM	NVIDIA's compiled kernels; peak throughput, most rigidity
llama.cpp	CPU + Apple Silicon + any GPU; GGUF; the edge/laptop king (Open-Weights handbook, Ch. 3)
Ollama	llama.cpp + model management; the developer default

All expose the OpenAI-compatible API (Open-Weights handbook, Ch. 5) — the engine choice is a deployment decision, invisible to clients.

6.2 Multi-GPU: sharding and replication

Strategy	Mechanism
tensor parallel (`-tensor-parallel-size`)	split each layer's matrices across GPUs; all GPUs busy per token; needs fast interconnect (NVLink)
pipeline parallel	split layers across GPUs; simpler, lower utilization, works over slower links
replication	whole model per GPU + a load balancer — simplest scale-out; combine with TP for big models
`device_map="auto"`	accelerate sharding for *inference on one box* (HF handbook, Ch. 2)

```
vllm serve meta-llama/Llama-3.3-70B-Instruct \   # 70B on 2x48GB: TP=2
    --tensor-parallel-size 2 --max-model-len 8192 \
    --quantization awq_marlin
```

6.3 The deployment patterns

1. **single GPU, one model**: vLLM + continuous batching + prefix caching; the workhorse.
2. **one big model, small cards**: TP sharding or quantization (AWQ/GPTQ, Open-Weights handbook Ch. 4).
3. **many models, one box**: multi-engine on separated GPUs with a router — or SGLang/vLLM's multi-model serving.

4. **fleet**: replicas behind a load balancer; autoscale on queue depth; prefix-cache-friendly routing (same system prompts to the same replica).
5. **edge**: llama.cpp/Ollama with GGUF Q4; the API stays identical.

PITFALL

Deployment traps: (1) TP without NVLink — tensor parallel communicates per token; over PCIe it crawls, use pipeline or replicate instead; (2) `-gpu-memory-utilization` left at 0.9 on a shared card — the KV pool starves other tenants (Ch. 2's arithmetic is yours to size); (3) benchmark numbers quoted at 1k context vs. your 32k workload — attention cost is quadratic in prefill; re-run `llama-bench`/`vllm bench` at your lengths.

THE ONE-SENTENCE SUMMARY

Engines are bundles of this handbook: continuous batching (Ch. 3), paged KV (Ch. 2), FlashAttention kernels (Ch. 5), speculative decoding (Ch. 4), behind one OpenAI-compatible endpoint. Choose by hardware and workload; benchmark at your context lengths; keep the client unaware.

APPENDIX A

Cheat Sheet

A.1 The cost model

Phase	Bound	Levers
prefill	compute (FLOPs)	FlashAttention, chunked prefill, prefix caching
decode	memory bandwidth ($2N$ bytes/token)	quantization, batching, speculative decoding
KV cache	capacity	GQA, paging, windows, prefix caching

A.2 KV cache arithmetic

```
kv_gb = (2 * layers * kv_heads * d_head * ctx * batch * dtype) / 1e9
# Llama-3-8B @ 8k ctx, bf16, batch 1:  ~1.1 GB
# GQA (8 kv heads) is the 4x lever; quantize KV for more
```

A.3 Engine flags

Dial	Flag
context cap	`-max-model-len`
Batch	`-max-num-seqs` (vLLM); continuous by default
Prefix cache	`-enable-prefix-caching`
Speculative	`-speculative-model` + `-num-speculative-tokens`
Sharding	`-tensor-parallel-size N`
KV pool	`-gpu-memory-utilization 0.9`
Chunked prefill	`-enable-chunked-prefill`

A.4 Speculative decoding

Piece	Note
acceptance	$\alpha \approx 0.7$–0.9 for same-family drafts
speedup	$(1-\alpha^{k+1})/(1-\alpha)$ tokens per target pass
lossless	exactly the target distribution (modified rejection sampling)
cheap drafts	same family smaller; Eagle/Medusa heads; prompt n-grams

A.5 Decision table

Situation	Stack
laptop / CPU	llama.cpp + GGUF Q4 (Open-Weights handbook)
one GPU, personal	Ollama or vLLM single stream
one GPU, traffic	vLLM, continuous batching, tune `max_num_seqs`
70B on 2 GPUs	vLLM TP=2 + AWQ/GPTQ
long prompts + many users	chunked prefill + prefix caching + SGLang radix cache
copy-heavy tasks	speculative decoding or prompt-lookup

THE HABITS THAT MATTER

Benchmark at your context lengths and your batch sizes. Measure TTFT and TPOT separately — they have different walls. Enable prefix caching before buying more GPUs. Try speculative decoding before upgrading hardware. And keep the OpenAI-compatible endpoint between you and the engine, so every optimization is a config line.